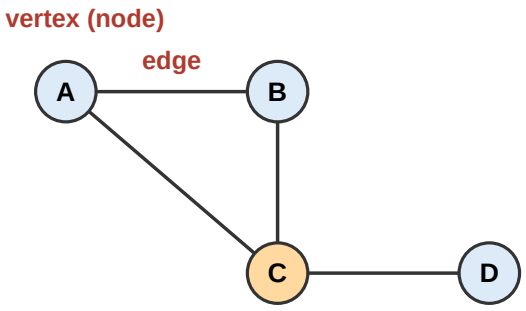
GRAPHS

Simple Notes: Basic Terms, Types of Graphs and Representation

1. What Is a Graph?

Answer: A **graph** is a non-linear data structure made of a set of **vertices** (nodes) and a set of **edges** that connect pairs of vertices. It is written as **G = (V, E)**, where V = set of vertices and E = set of edges.



Neighbours of C: A, B, D

(they are joined to C by an edge)

Degree of C = 3

(C has 3 edges)

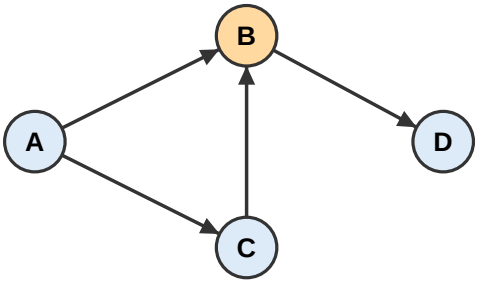
Figure 1: A graph with 4 vertices (A, B, C, D) and 4 edges

Basic terms

Term	Definition
Node / Vertex	A point (object) in the graph. **Node** and **vertex** mean the same thing. Example: A, B, C, D.
Edge	A line that connects two vertices. Example: A-B, C-D.
Neighbour (adjacent vertex)	Two vertices that are joined directly by an edge are neighbours. Neighbours of C are A, B and D.
Degree	The number of edges connected to a vertex (in an undirected graph). Degree of C = 3.
In-degree	In a directed graph: the number of edges **coming into** a vertex.
Out-degree	In a directed graph: the number of edges **going out** of a vertex.

Vertex	A	B	C	D
Neighbours	B, C	A, C	A, B, D	C
Degree	2	2	3	1

In-degree and out-degree



Directed graph (arrows)

For vertex B (orange):
edges coming IN: A>B, C>B = 2
edges going OUT: B>D = 1

In-degree(B) = 2, Out-degree(B) = 1

Figure 2: In a directed graph, arrows show the direction of each edge

Vertex	A	B	C	D
In-degree (arrows coming in)	0	2	1	1
Out-degree (arrows going out)	2	1	1	0

Check: total in-degree = total out-degree = number of edges (4).

2. Types of Graphs

A. Based on edges: Undirected and Directed

Answer: Undirected graph: a graph whose edges have **no direction**. Edge A-B is the same as B-A, so you can go both ways (like a two-way road).

Answer: Directed graph (digraph): a graph whose edges have a **direction**, shown by arrows. Edge A→B lets you go only from A to B (like a one-way road).

Undirected graph **Directed graph**

Figure 3: Undirected graph (plain lines) and directed graph (arrows)

B. Based on weight: Weighted and Unweighted

Answer: Weighted graph: a graph in which every edge has a number called its **weight** (such as distance, cost or time).

Answer: Unweighted graph: a graph in which edges have **no weights**; all edges are treated as equal.

Unweighted graph **Weighted graph**

Figure 4: Unweighted graph and weighted graph

C. Cyclic and Acyclic graphs

A **cycle** is a path that starts and ends at the **same vertex** (for example A-B-C-A).

Answer: Cyclic graph: a graph that contains **at least one cycle**.

Answer: Acyclic graph: a graph that contains **no cycle**. (A directed acyclic graph is called a **DAG**; a tree is also an acyclic graph.)

Cyclic (has cycle A-B-C-A) **Acyclic (no cycle)**

Figure 5: Cyclic graph (cycle in red) and acyclic graph

Quick summary

Based on	Types	Remember as
Edges	Undirected / Directed	Plain lines / Arrows
Weight	Weighted / Unweighted	Edges have numbers / No numbers
Cycle	Cyclic / Acyclic	Has a cycle / No cycle

3. Representation of Graphs

A graph is stored in a computer in two main ways: the **adjacency matrix** and the **adjacency list**. Both are shown for the graph in Figure 1 (undirected) and the graph in Figure 2 (directed).

A. Adjacency matrix

Answer: An **adjacency matrix** is a V x V table (2D array). The entry in row i and column j is **1** if there is an edge from vertex i to vertex j, otherwise **0**.

Undirected graph (Figure 1):

	A	B	C	D
A	0	1	1	0
B	1	0	1	0
C	1	1	0	1
D	0	0	1	0

Directed graph (Figure 2): read as *row = from, column = to*.

	A	B	C	D
A	0	1	1	0
B	0	0	0	1
C	0	1	0	0
D	0	0	0	0

- For an **undirected** graph the matrix is **symmetric** (same on both sides of the diagonal).
- For a **directed** graph it is usually **not symmetric**.
- For a **weighted** graph, write the **weight** instead of 1.

Advantage	Disadvantage
Easy to understand; checking an edge is quick	Uses V x V memory, even when there are few edges

B. Adjacency list

Answer: An **adjacency list** stores, for each vertex, a **list of its neighbours**. The graph is an array of such lists.

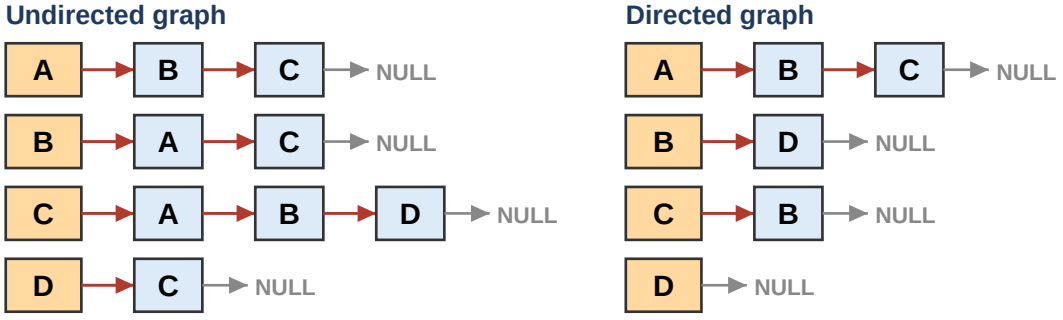


Figure 6: Adjacency lists of the undirected graph and the directed graph

- For a **directed** graph, a vertex's list holds only the vertices it points **to** (D has no outgoing edge, so its list is empty).
- For a **weighted** graph, store the weight with each neighbour, e.g. (B, 4).

Advantage	Disadvantage
Uses less memory (only the existing edges); good for graphs with few edges	Checking whether a particular edge exists is slower

Matrix vs List

Point	Adjacency matrix	Adjacency list
Structure	V x V table	List of neighbours for each vertex
Memory	V x V (more)	V + E (less)
Check an edge	Very quick	Slower
Best for	Graphs with many edges	Graphs with few edges

Quick Revision

1. Graph = set of vertices + set of edges, G = (V, E).
2. Node and vertex are the same; an edge joins two vertices.
3. Neighbours: vertices joined directly by an edge.
4. In-degree = arrows coming in; out-degree = arrows going out.
5. Undirected: no direction. Directed: arrows.
6. Weighted: edges have numbers. Unweighted: no numbers.
7. Cyclic: has a cycle. Acyclic: no cycle.
8. Adjacency matrix: V x V table of 0 and 1. Adjacency list: neighbours of each vertex.

Likely exam questions

1. Define a graph. What are vertex, edge and neighbour? (3 marks)
2. Define in-degree and out-degree with an example. (3 marks)
3. Explain undirected and directed graphs with diagrams. (3 marks)
4. Explain weighted and unweighted graphs with diagrams. (3 marks)
5. What are cyclic and acyclic graphs? (2 marks)
6. Explain adjacency matrix and adjacency list with an example. (5 marks)